Architecture as Code (AaC)

1. Introduction

Architecture as Code treats architectural knowledge the same way modern teams treat source code, infrastructure-as-code scripts, and automated tests: as living artefacts that stay in version control, participate in CI/CD pipelines, and can be regenerated on demand. Instead of relying on slide decks or wikis that drift out of date, the architecture becomes executable documentation that evolves hand in hand with the system it describes.

This guide explains the idea, covers the history that led to it, describes how it is realised inside the Specify toolkit, and offers practical steps for teams that want to adopt AaC.

2. Why Architecture as Code?

Architecture documents have often been:

- **Static**: diagrams captured once, never updated.
- **Detached**: living in tools outside the code repo, making them easy to forget.
- **Unactionable**: providing little link between design intent and enforcement.

Architecture as Code addresses these issues by ensuring that:

1. **Architecture lives with the code**. Every change shows up in diffs and pull requests; reviewers catch misalignment early.
2. **Automation becomes possible**. Scripts and CI jobs validate diagrams, export PDFs, and block changes that miss mandatory metadata (for example, missing security notes or test suites).
3. **Traceability improves**. Every architectural decision, ownership change, or dependency update carries a commit, author, and rationale.
4. **Security and compliance tighten**. Because artefacts are machine-readable, you can enforce policies automatically (for example, refusing a merge if a container lacks a security checklist).
5. **AI assistants operate safely**. When the architecture model is structured, AI can regenerate diagrams, keep breadcrumbs up to date, and maintain the tests overview without working blindly.

3. A Short History

1. **1990s 2000s: Document-driven architecture**
    - Architecture lived in UML and Word documents.
    - Updates were manual and rarely synchronised with the code base.
2. **2006 2015: Infrastructure as Code influence**
    - Tools like Terraform and CloudFormation proved configuration can be versioned.
    - Architecture artefacts, however, often remained informal.
3. **2016 2020: Diagrams as Code**

    - Libraries such as Mermaid, PlantUML, Graphviz, and Structurizr allowed diagrams to be generated from text or code.
    - Teams gained repeatable diagrams but still struggled to keep narrative data (owners, security, risk) aligned.

4. **2021 today: Architecture as Code**

    - Structured models (JSON/YAML) plus templated documentation produce full architecture packets.
    - Linting, CI checks, and AI assistants keep architecture in lock-step with implementation and tests.

4. C4 as the Backbone

The Specify toolkit adopts Simon Brown's C4 model as the descriptive framework for architecture-as-code. C4 breaks documentation into four zoom levels:

- **System Context (C1)**: Who interacts with the system and why.
- **Container (C2)**: Which deployable/runtime units exist and how they communicate.
- **Component (C3)**: How responsibilities split inside a container.
- **Code (C4)**: Optional details (modules, classes, database schema) for critical components.

Brief Timeline of C4

- **2011**: Simon Brown begins presenting C4 as a simpler alternative to heavyweight UML.
- **2012 2014**: Structurizr and related tooling arrive, enabling diagrams to derive from code and documentation templates.
- **2016**: "Diagram as code" gains steam with Mermaid and PlantUML, fitting C4's intent perfectly.
- **2020+**: C4 becomes a common standard for architecture documentation, especially when combined with architecture-as-code pipelines.

In Specify, each C-level has a Markdown template with:

- navigation breadcrumbs,
- Mermaid scaffolds for diagrams,
- security and database sections,
- links to related views (container -> components, components -> code structure).

5. Core Principles of Architecture as Code

1. **Model-as-data**: `specs/architecture/architecture.json` is the single source of truth (systems, containers, components, tests, change log). Every Markdown view derives from it.
2. **Views are projections**: C1-C7 Markdown files project the model into human-friendly reports.
3. **Version discipline**: Specify enforces a patch-only scheme (`1.0.x`) so view versions stay aligned with the model.
4. **Executable gates**: commands (`/specify.architecture.create`, `/specify.architecture.update`) and tools (`mmdc`, `md-to-pdf`) automate validation and delivery.
5. **Testing and security baked in**: the C7 tests overview lists every suite (purpose, owners, commands, coverage focus, risks, last run). Component/container templates require security notes and breadcrumb links to code structure.

6. **Tool-agnostic**: Markdown + JSON play well with any CI, doc site, or knowledge base.

6. Architecture as Code in the Specify Workflow

1. **Foundations (`/specify.constitution`)**
    - Captures non-negotiables (quality, security, delivery) in `.specify/memory/constitution.md`.
    - Every downstream command references these rules.
2. **Create the architecture (`/specify.architecture.create`)**
    - Generates `architecture.json`, C1C7 Markdown with breadcrumbs, diagrams, and cross-links.
    - Seeds `c7_tests/tests_overview.md` as the exhaustive suite catalogue.
    - Encourages `mmdc` validation and PDF export.
3. **Update the architecture (`/specify.architecture.update`)**
    - Applies patch-only version bumps, regenerates views, records changes in `architecture_logs.md`.
    - Ensures components link to code structure and the tests table remains accurate.
4. **Specification and planning (`/specify.specify`, `/specify.plan`, `/specify.tasks`)**
    - Reference the architecture model directly.
    - Changes trigger architecture updates so requirements, plan, and architecture stay synchronised.
5. **Implementation (`/specify.implement`, optional `/specify.clarify`, `/specify.analyze`)**
    - Code changes must refresh breadcrumbs, Mermaid diagrams, security notes, and the tests overview.
6. **Tooling checklist**
    - `python -m compileall src/specify_cli`
    - `npm install -g md-to-pdf @mermaid-js/mermaid-cli`
    - `./specs/architecture/export_to_pdf.sh`
    - `mmdc -i path/to/file.md -o /tmp/file.svg`

7. Case Study (Illustrative)

Imagine a workflow orchestrator platform:

1. `architecture.json` lists the orchestrator service, document gateway, flow manager, and their relationships.
2. The C1 system-context view highlights external actors (operations staff, third-party services) and the business goals they achieve through the platform.
3. The C2 view shows the REST API, worker containers, databases, and message queues. Mermaid diagrams depict interactions.
4. The C3 view zooms into the API container: components such as SchedulerService, DocumentFacade, FlowPolicyEngine, each with security notes and linked code structure.

5. The C4 view documents key modules and database schema for FlowPolicyEngine (Mermaid class diagram + ER diagram scaffold).
6. The C5 dynamic view captures a typical orchestration flow (user trigger -> API -> flow engine -> worker).
7. The C6 deployment view shows staging/production clusters, edge gateways, databases, and internal networks.
8. The C7 tests overview lists suite IDs (`svc-orders-unit`, `api-gateway-contract`, `ui-end-to-end`), commands, coverage, status, and known gaps.
9. A change to FlowPolicyEngine triggers `/specify.architecture.update`: the patch version increments, the component diagram refreshes, database schema notes update, and the tests table adds a new flow-policy-contract suite entry.
10. The team runs `mmdc` to check Mermaid syntax and `export_to_pdf.sh` to deliver the updated architecture packet.

8. Adoption Checklist

- [ ] Constitution captured and stored in `.specify/memory/constitution.md`.
- [ ] `/specify.architecture.create` executed and `architecture.json` committed.
- [ ] C1-C7 views contain breadcrumbs, Mermaid diagrams, security notes, and cross-links.
- [ ] `c7_tests/tests_overview.md` lists every suite with owners, commands, coverage focus, status, risks, last run.
- [ ] `mmdc` used to validate diagrams; `export_to_pdf.sh` run as needed.
- [ ] Patch versioning (`1.0.x`) respected; updates recorded in `architecture_logs.md`.

9. Next Steps

1. Run `/specify.constitution` and `/specify.architecture.create` on a project.
2. Wire Mermaid validation (`mmdc`) and PDF export into local scripts or CI.
3. Review architecture updates the same way you review code: pull requests, inline comments, automated checks.
4. Encourage developers and architects to update architecture artefacts with every meaningful change.
5. Monitor the change log and tests overview to ensure coverage and compliance stay intact.

10. References and Further Reading

- Simon Brown, *Software Architecture for Developers* introduction to C4.
- Mermaid documentation https://mermaid.js.org/
- md-to-pdf CLI https://github.com/simonhaenisch/md-to-pdf
- Architecture as Code blog posts (various) search for "architecture as code" discussions from ThoughtWorks, InfoQ, and community talks.

By codifying architecture you ensure that diagrams, decisions, and controls evolve with your system, are reviewed like code, and drive the entire development lifecycle. Architecture as Code is the backbone that keeps specifications, plans, tasks, tests, and implementation permanently aligned.